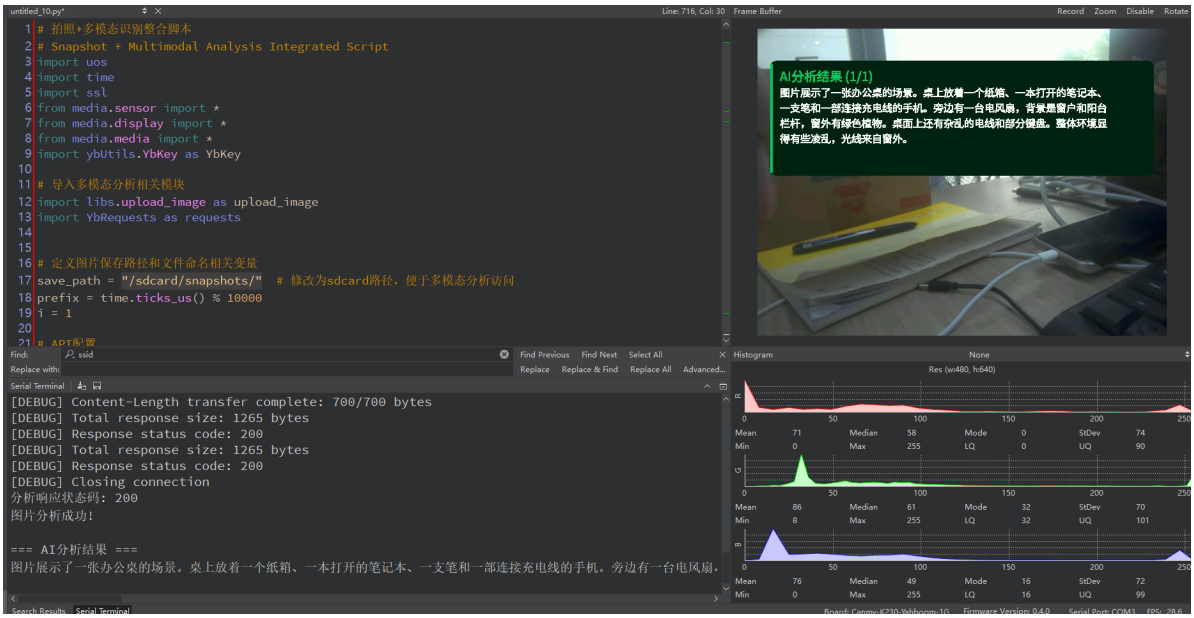
# 图片理解-场景理解

**图片理解-场景理解**
- 例程效果
- 代码讲解
  1. 导入必要的模块
  2. 定义全局变量和常量
  3. 辅助函数
  4. 主程序逻辑
  5. 异常处理与资源释放

## 例程效果

本节例程源码文件名为 `photo_ai_analysis.py`。

连接IDE，在IDE中运行代码，你将看到一个实时相机预览界面。按下按键时，程序会捕获照片，自动连接WiFi（首次使用），上传图片并使用AI进行多模态分析（如图像描述）。分析结果以分页方式显示在屏幕上




## 代码讲解

### 1. 导入必要的模块

```
import uos
import time
import ssl
from media.sensor import *
from media.display import *
from media.media import *
import ybUtils.YbKey as YbKey
import libs.upload_image as upload_image
import YbRequests as requests
```

- `uos`：用于操作系统相关操作，如目录管理。
- `time`：用于时间控制和延时。
- `ssl`：支持安全套接字层，用于HTTPS请求。
- `media.sensor`，`media.display`，`media.media`：用于传感器控制、显示和媒体管理。
- `ybUtils.YbKey`：用于按键检测。
- `libs.upload_image`：自定义模块，用于图像上传。
- `YbRequests`：用于发送HTTP请求。

## 2. 定义全局变量和常量

```
save_path = "/sdcard/snapshots/"
prefix = time.ticks_us() % 10000
i = 1
API_KEY = ""
WIFI_SSID = ""
WIFI_KEY = ""
network_connected = False
UI_COLORS = { ... }
```

- `save_path`：照片保存路径。
- `prefix`，`i`：用于生成文件名。
- `API_KEY`，`WIFI_SSID`，`WIFI_KEY`：API密钥和WiFi配置（需用户提供）。
- `network_connected`：布尔值，记录网络连接状态。
- `UI_COLORS`：字典，定义界面颜色方案。

## 3. 辅助函数

- `draw_rounded_rect()`：绘制圆角矩形。
- `draw_progress_bar()`：绘制进度条，支持文本显示。
- `show_progress_with_animation()`：显示带动画的进度界面。
- `draw_status_card()`：绘制状态卡片。
- `show_loading_animation()`：显示加载动画。
- `enhanced_network_connection()`：增强的WiFi连接函数，支持进度显示和重试。
- `enhanced_ai_analysis()`：增强的AI分析函数，支持重试、网络错误处理和UI反馈。
- `smart_text_processing()`：智能处理文本，提取关键信息。
- `split_text_to_pages()`：将文本分页以适应显示。
- `enhanced_display_analysis_result()`：显示分析结果，支持分页和动画。
- `analyze_image_with_dashscope()`：使用DashScope API分析图像。
- `ensure_dir()`：确保目录存在。

## 4. 主程序逻辑

```
if __name__ == "__main__":
    try:
        key = YbKey.YbKey()
        sensor = Sensor()
        sensor.reset()
        sensor.set_framesize(width=640, height=480, chn=CAM_CHN_ID_1)
        sensor.set_pixformat(Sensor.RGB565, chn=CAM_CHN_ID_1)
        Display.init(Display.ST7701, width=640, height=480, to_ide=True)
        MediaManager.init()
        sensor.run()

        last_status = False

        while True:
            img = sensor.snapshot(chn=CAM_CHN_ID_1)
            img2 = image.Image(640, 480, image.RGB565)
            img2.clear()
            img2.copy_from(img)
            draw_status_card(img2, 20, 20, 600, 80, f"K230 & 场景分析 - 照片 {i}",
f"网络状态: {'🌐 已连接' if network_connected else '📶 未连接'}\n按键进行拍照和AI分析",
'info')
            img2.draw_string_advanced(50, 420, 20, "按下按键开始拍照",
color=UI_COLORS['primary'])
            img2.draw_string_advanced(50, 450, 16, "首次使用将自动连接WiFi",
color=UI_COLORS['text_dim'])
            Display.show_image(img2, 0, 0, Display.LAYER_OSD2)

            if key.is_pressed() == 1:
                if not last_status:
                    last_status = True
                    ensure_dir(save_path + str(prefix) + "/")
                    image_path = save_path + str(prefix) + "/" + str(i) + ".jpg"
                    img.save(image_path)
                    if not network_connected:
                        ip = enhanced_network_connection(WIFI_SSID, WIFI_KEY,
img)
                        if ip:
                            network_connected = True
                    if network_connected:
                        result = enhanced_ai_analysis(API_KEY, image_path, "请用简
洁的语言描述这张图片中的主要内容，控制在200字以内", img)
                        enhanced_display_analysis_result(img, result['choices']
[0]['message']['content'], 8000)
                    i += 1
            else:
                last_status = False
    except KeyboardInterrupt as e:
        print(f"用户停止程序")
    except BaseException as e:
        print(f"发生异常: '{e}'")
    finally:
        sensor.stop()
        Display.deinit()
        uos.exitpoint(uos.EXITPOINT_ENABLE_SLEEP)
        time.sleep_ms(100)
```

```
        MediaManager.deinit()
```

- 初始化传感器、显示和媒体。
- 循环捕获图像并显示UI。
- 按键触发拍照、保存、分析和结果显示。
- 首次运行时自动连接WiFi。
- 异常处理：清理资源。

## 5. 异常处理与资源释放

- 在主程序中使用 `try-except-finally` 块处理异常，确保传感器和显示器资源被释放。